

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS
Facultad de Ciencias Puras · Carrera de Ingeniería Informática
Potosí — Bolivia

INF 220 — PROGRAMACIÓN ORIENTADA A OBJETOS Y ESTRUCTURAS DE DATOS

GUÍA DE LABORATORIO N.º 5

Cadenas y manejo de texto

Unidad	Unidad 1 — Programación Orientada a Objetos
Tema	Clase String e inmutabilidad · comparación de cadenas · métodos de manipulación · StringBuilder · formateo
Requisito	Guía de Laboratorio N.º 4 (métodos)
Stack	JDK Temurin 25 (LTS) · Visual Studio Code · Extension Pack for Java
Duración	1 sesión de laboratorio
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán

1 Objetivos

► Objetivo general

Manipular texto en Java empleando la clase String y StringBuilder, aplicando sus métodos de búsqueda, extracción, transformación y formateo para resolver problemas de procesamiento de cadenas.

► Objetivos específicos

- Comprender que String es inmutable y qué implica esa característica.
- Aplicar los métodos más comunes de String para manipular texto.
- Comparar cadenas de forma correcta distinguiendo equals de ==.
- Usar StringBuilder para construir y modificar texto de manera eficiente.
- Dar formato a la salida con printf y String.format.

2 Requisitos previos

Software	JDK Temurin 25 + Visual Studio Code con Extension Pack for Java.
Conocimientos	Definición y uso de métodos, tipos de datos, arreglos y control de flujo (Guías 1 a 4).

3 Marco teórico

► La clase String

En Java el texto se representa con la clase `String`. Aunque se usa como si fuera un tipo básico, es un **objeto**. Su característica más importante es que es **inmutable**: una vez creado, su contenido no cambia.

DEFINICIÓN · Inmutabilidad

Los métodos como `toUpperCase()` o `replace()` no modifican la cadena original: **devuelven una cadena nueva**. Si quieres conservar el resultado, debes asignarlo a una variable.

► Comparar cadenas

Este es uno de los errores más frecuentes. El operador `==` compara *referencias* (si son el mismo objeto), mientras que el método `equals()` compara el *contenido*.

IMPORTANTE · Compara siempre con `equals()`

Para saber si dos cadenas tienen el mismo texto usa `a.equals(b)`, nunca `a == b`. Para ignorar mayúsculas/minúsculas, usa `equalsIgnoreCase()`.

► Métodos más usados de String

<code>length()</code>	Cantidad de caracteres de la cadena.
<code>charAt(i)</code>	Carácter en la posición <code>i</code> (empieza en 0).
<code>substring(a, b)</code>	Subcadena desde <code>a</code> hasta <code>b-1</code> .
<code>indexOf(txt)</code>	Posición de la primera aparición de <code>txt</code> (o -1 si no está).
<code>replace(a, b)</code>	Devuelve la cadena con <code>a</code> reemplazado por <code>b</code> .
<code>toUpperCase()</code> / <code>toLowerCase()</code>	Convierte a mayúsculas o minúsculas.
<code>trim()</code>	Elimina espacios al inicio y al final.
<code>split(sep)</code>	Divide la cadena en un arreglo usando el separador <code>sep</code> .

► StringBuilder: texto mutable

Como `String` es inmutable, concatenar muchas veces con `+` dentro de un bucle crea muchos objetos temporales y es ineficiente. `StringBuilder` es una cadena **modificable** pensada para construir texto paso a paso con `append()`, `insert()` o `reverse()`.

► Formateo de salida

Con `printf()` y `String.format()` se controla cómo se muestra la información mediante especificadores:

<code>%d</code>	Número entero.
<code>%f</code> · <code>%.2f</code>	Número real; <code>%.2f</code> fija dos decimales.
<code>%s</code>	Cadena de texto.
<code>%-10s</code> · <code>%5d</code>	Ancho y alineación (izquierda con el signo -).

%n

Salto de línea (portable entre sistemas).

4 Desarrollo de la práctica

Trabaja en una carpeta Lab05.

► Actividad 1 — Métodos esenciales de String

Explora los métodos más comunes sobre una misma cadena:

```
String texto = " Programación en Java ";

System.out.println("Longitud: " + texto.length());
System.out.println("Sin espacios: " + texto.trim());
System.out.println("Mayúsculas: " + texto.toUpperCase());
System.out.println("Posición de 'Java': " + texto.indexOf("Java"));
System.out.println("Reemplazo: " + texto.replace("Java", "POO"));
System.out.println("Subcadena: " + texto.trim().substring(0, 12));
```

EJEMPLO · Encadenar métodos

Fíjate en `texto.trim().substring(0, 12)`: como cada método devuelve una nueva cadena, se pueden encadenar llamadas una tras otra.

► Actividad 2 — Comparar cadenas correctamente

Comprueba la diferencia entre `==` y `equals()`:

```
String a = "hola";
String b = "hola";
String c = new String("hola");

System.out.println(a == b);           // true (mismo objeto del pool)
System.out.println(a == c);           // false (objeto distinto)
System.out.println(a.equals(c));       // true (mismo contenido)
System.out.println(a.equalsIgnoreCase("HOLA")); // true
```

NOTA · El pool de cadenas

Java reutiliza los literales de texto iguales en un «pool», por eso `a == b` puede dar `true`. Pero nunca confíes en eso para comparar: usa siempre `equals()`.

► Actividad 3 — Recorrer una cadena

Recorre carácter por carácter con `charAt()` para contar las vocales de una frase:

```
String frase = "Estructuras de Datos";
int vocales = 0;
String v = "aeiouAEIOU";

for (int i = 0; i < frase.length(); i++) {
    char c = frase.charAt(i);
    if (v.indexOf(c) >= 0) {
        vocales++;
    }
}
```

```
System.out.println("Vocales: " + vocales);
```

► Actividad 4 — Dividir texto con split

Separa una línea de datos en sus partes usando split():

```
String registro = "Ana,20,78.5";
String[] partes = registro.split(",");

System.out.println("Nombre:  " + partes[0]);
System.out.println("Edad:    " + partes[1]);
System.out.println("Promedio: " + partes[2]);
```

► Actividad 5 — Construir texto con StringBuilder

Construye y transforma texto de forma eficiente:

```
StringBuilder sb = new StringBuilder();
sb.append("Hola");
sb.append(" ");
sb.append("mundo");
sb.insert(0, ">> ");

System.out.println(sb.toString());           // >> Hola mundo
System.out.println(sb.reverse().toString()); // odnum aloH <<
```

IMPORTANTE · Por qué StringBuilder

Para concatenar dentro de un bucle largo, StringBuilder es mucho más eficiente que usar + sobre un String, porque evita crear una cadena nueva en cada vuelta.

► Actividad 6 — Formatear la salida

Da formato con printf() y arma una línea con String.format():

```
String nombre = "Ana";
double promedio = 78.456;

System.out.printf("%s obtuvo %.2f%n", nombre, promedio);

String linea = String.format("%-10s | %.2f", nombre, promedio);
System.out.println(linea);
```

5 Bibliografía y recursos

- Deitel, P. & Deitel, H. — Cómo programar en Java. Pearson.
- Sierra, K. & Bates, B. — Head First Java. O'Reilly.
- Clase String (API oficial):
<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html>